



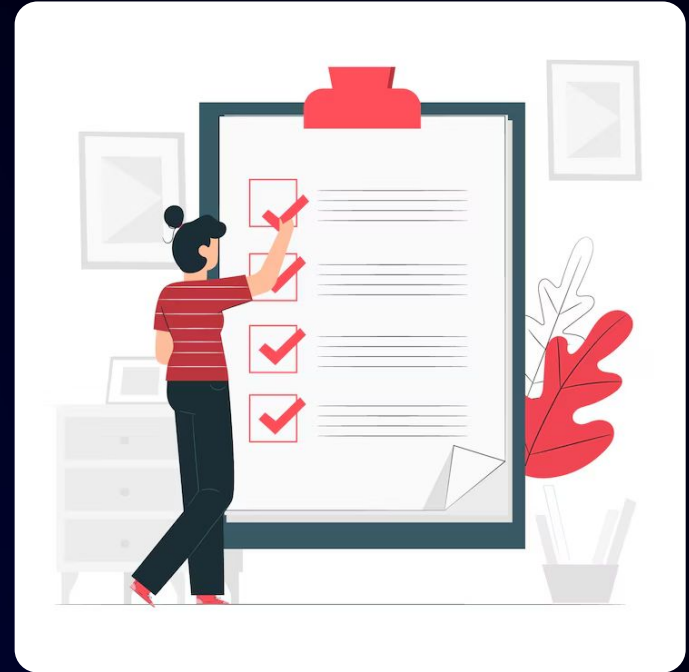
Introduction to JavaScript & Basics

Learning Objectives

- Understand how JavaScript adds logic and interactivity to web pages
- Identify where JavaScript runs (browser & Node.js)
- Learn basic data types (number, string, boolean, null, undefined)
- Understand the purpose of variables and how to use them
- Differentiate between let, const, and var
- Learn the process: declare → assign → print
- Understand identifiers and naming rules
- Write and use comments effectively in code
- Apply arithmetic and comparison operators
- Understand the difference between == and ===

Prerequisites

- Basic understanding of how web pages work (HTML & CSS)
- Familiarity with using a web browser (Chrome, Edge, etc.)
- Basic knowledge of computers and programming concepts
- Ability to use a code editor (VS Code or similar)
- Willingness to learn logic and problem-solving basics





What is JavaScript

Introduction to Javascript

- **JavaScript (JS)** makes websites interactive
- Runs in browsers and on servers (Node.js)
- Works with **HTML (structure)** and **CSS (style)**
- Dynamically typed (no need to declare data types)



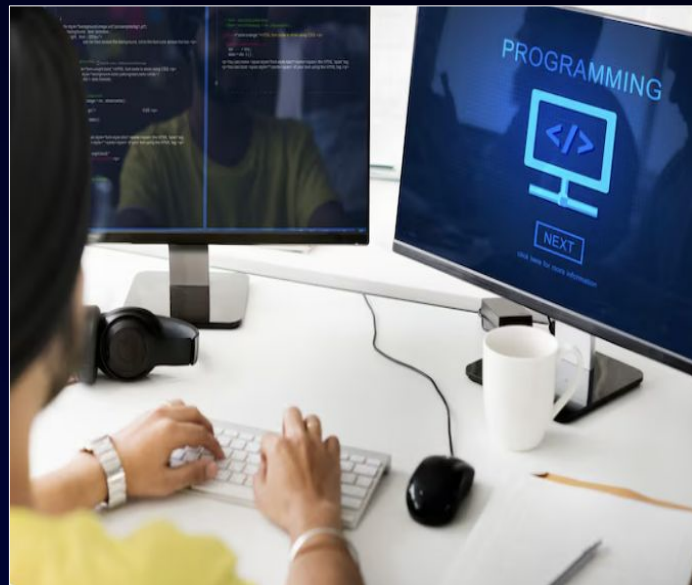
How does JavaScript add interactivity?

- Handles user actions (clicks, typing, scrolling)
- **Updates** content dynamically without reload
- Uses logic (conditions, loops, functions)
- Validates user input
- Manipulates **HTML & CSS (DOM)**
- Adds animations and effects



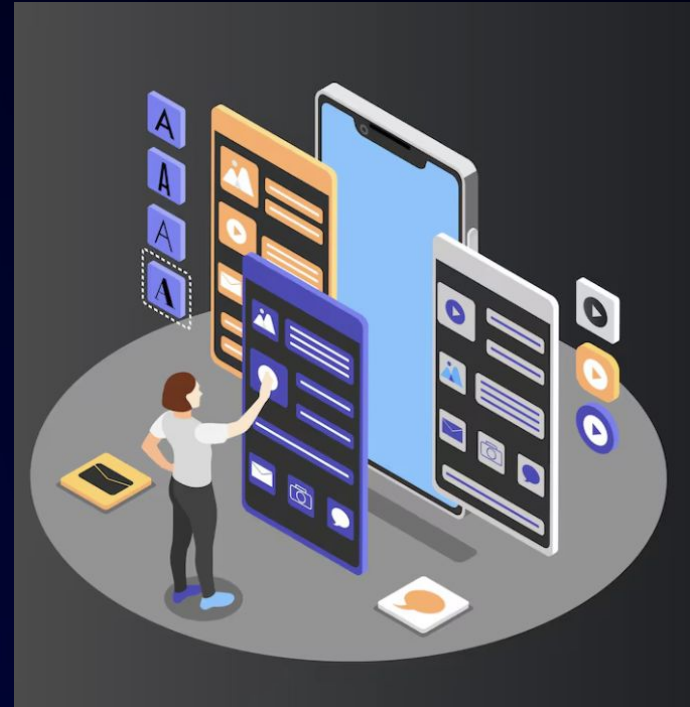
Where does JavaScript execute?

- Runs in the **browser (client-side)** for interactivity
- Uses the browser's **JavaScript engine** (e.g., V8)
- Can run on the **server-side** with Node.js
- Works across **multiple browsers**
- Supports both **frontend and backend** development



Real-World Use Cases :

- **Button Click (Browser):**
User clicks a button → JS shows a popup message
- **Form Validation (Logic + Browser):**
User enters email → JS checks if valid before submitting
- **API Call (Frontend ↔ Backend):**
JS fetches data from server and displays it on page
- **Cross-Browser:**
Same interactive website works on Chrome, Firefox, Edge



Pop Quiz

Where does JavaScript primarily run to make web pages interactive?

A

Server only

B

Browser (client-side)

Pop Quiz

Where does JavaScript primarily run to make web pages interactive?

A

Server only

B

Browser (client-side)



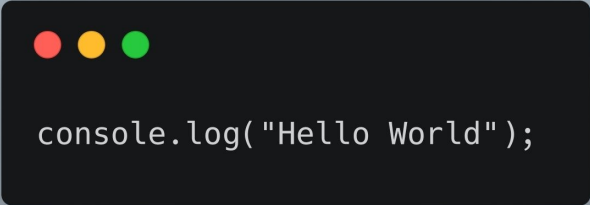
Node.js Setup & Running JS

Verify Node.js Installation

- Open terminal / command prompt
- Run command: **node -v**
- Displays installed Node.js version
- If version appears → ☒ Node.js is installed
- If error shows → ☐ Node.js not installed properly



Simple code with Node.js

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a single line of Node.js code.

```
console.log("Hello World");
```

Pop Quiz

Which command is used to check if Node.js is installed?

A

node start

B

node -v

Pop Quiz

Which command is used to check if Node.js is installed?

A

node start

B

node -v



Values and Data Types

What is a Value in Programming?

- A **value** is any piece of data stored in a program
- It represents **information used in code**
- Can be of different **types** (number, string, boolean, etc.)
- Used in **variables, operations, and expressions**
- Example: 10, "Hello", true
- Values help programs **process and produce results**



Data Types in JavaScript

- **Definition : Data types** define the kind of values a variable can hold and how they are used in a program.

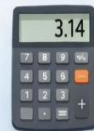
Types in JavaScript :

- **Number** : Numeric values (e.g., 10, 3.14)
- **String** : Text data (e.g., "Hello")
- **Boolean** : Logical values (true, false)
- **Null** : Intentional absence of value (mention)
- **Undefined** : Variable declared but not assigned (mention)

Common Data Types in JavaScript

Data types define the kind of values a variable can hold and how they are used in a program.

Number



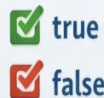
Numeric Values
10, 3.14

String



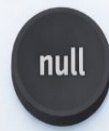
Text Data
"Hello"

Boolean



Logical Values
true, false

Null



Absence of Value
(Intentional Empty)

Undefined



No Assigned Value
(Variable Unset)

Data Type Examples

Number:

```
let age = 25;
```

String:

```
let name = "John";
```

Boolean:

```
let isLoggedIn = true;
```

Null:

```
let data = null;
```

Undefined:

```
let value;
```



Pop Quiz

Which data type represents true or false values in JavaScript?

A

String

B

Boolean

Pop Quiz

Which data type represents true or false values in JavaScript?

A

String

B

Boolean



Variables (let, const, var)

Why Do We Use Variables?

- Store **data values** for use in a program
- Help **reuse data** without repeating values
- Make code **more readable and organized**
- Allow data to be **updated or changed**
- Used in **calculations and logic operations**
- Act as **containers** for storing information



Ways to Declare Variables in JavaScript

let : Used for variables that can **change** later

const : Used for variables that **cannot be reassigned (fixed)**

var : Older way of declaring variables (avoid using, legacy)

Note : Prefer let & const in modern JavaScript



How It Works

```
// Declare the variable :  
let name;
```

```
// Assign the value :  
name = "John";
```

```
// Print The statement :  
console.log(name);
```



Pop Quiz

Which step comes first in the process: declare → assign → print?

A

Assign

B

Declare

Pop Quiz

Which step comes first in the process: declare → assign → print?

A

Assign

B

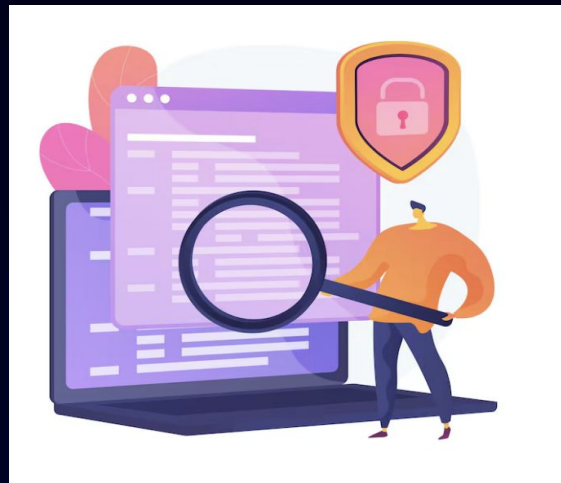
Declare



Identifiers & Naming Rules

What are Identifiers in JavaScript?

- **Identifiers** are the names given to variables
- Used to **identify and access** data in a program
- Must start with a **letter, _ (underscore), or \$**
- Cannot start with a **number**
- Should not use **reserved keywords** (e.g., let, const)
- Should be **meaningful and descriptive** for better readability
- Example: `let userName = "John";`



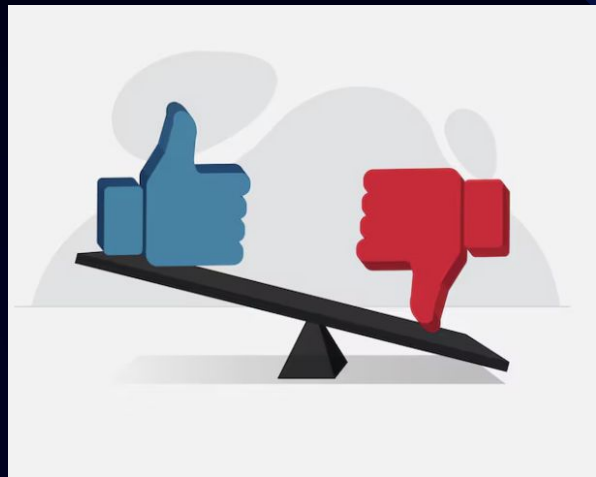
Rules for Naming Identifiers

- **Cannot start with a number** : must begin with a letter, `_`, or `$`
- **Cannot contain spaces** : use camelCase or underscores instead
- **Are case-sensitive** : name and Name are treated as different variables
- **Follow proper naming conventions** : keep names clear, meaningful, and easy to understand



Good vs Bad Identifier Examples

-  Good: `userName` : clear and meaningful
-  Bad: `123name` : starts with a number
-  Good: `totalMarks` : no spaces, easy to read
-  Bad: `total marks` : contains space
-  Good: `isLoggedIn` : descriptive and readable
-  Bad: `x` : not meaningful
-  Good: `priceValue`
-  Bad: `Pricevalue` : inconsistent casing



Pop Quiz

Which of the following is a valid identifier name in JavaScript?

A

123name

B

userName

Pop Quiz

Which of the following is a valid identifier name in JavaScript?

A

123name

B

userName



Comments

Why Do We Use Comments in Code?

- Used to **explain code** for better understanding
- Improve **readability for developers**
- Help in **debugging and maintaining code**
- Allow adding **notes or reminders**
- Ignored by the JavaScript engine (**not executed**)
- Useful for team collaboration



Types of Comments in JavaScript

- Single-line comment (//) : Used to write a comment in one line
- **Example:** `// This is a single-line comment`
- Multi-line comment (/* */) : Used to write comments across multiple lines

Example:

```
/* This is a  
multi-line comment */
```

- Comments are used to explain code and improve readability



Pop Quiz

Which symbol is used for single-line comments in JavaScript?

A

`//`

B

`/* */`

Pop Quiz

Which symbol is used for single-line comments in JavaScript?

A

//

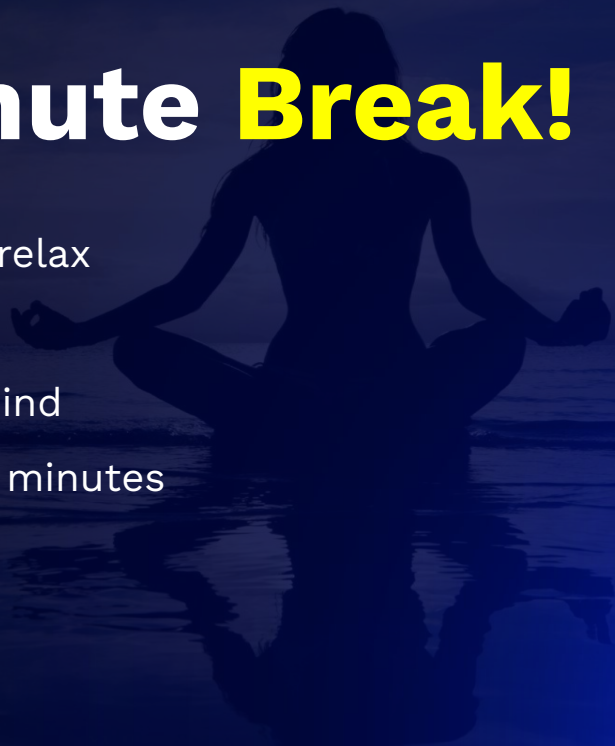
B

/* */



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes





Basic Operators

Arithmetic Operators in JavaScript

Used to perform mathematical operations on values

:

Addition (+) : Adds two values

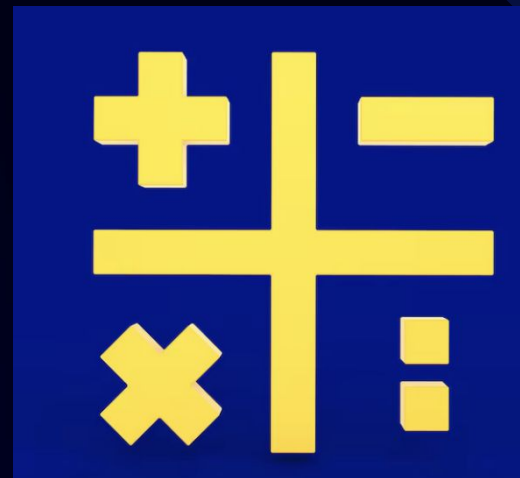
Subtraction (-) : Subtracts one value from another

Multiplication (*) : Multiplies values

Division (/) : Divides one value by another

Modulus (%) : Gives the remainder

Help in calculations and expressions



comparison operators in javascript

Used to compare two values :

Equal (==) : Checks if values are equal

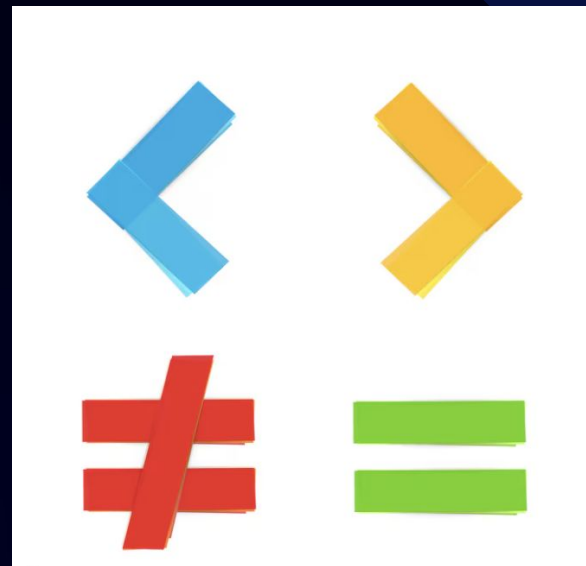
Strict Equal (===) : Checks value and type

Not Equal (!=) : Checks if values are not equal

Greater Than (>) : Checks if one value is larger

Less Than (<) : Checks if one value is smaller

Returns a boolean value (true or false)



Difference Between == and ===

== (Loose Equality) : Compares values after type conversion

Example: `5 == "5"` :  true

=== (Strict Equality) : Compares both value and data type

Example: `5 === "5"` :  false

Use `===` for more accurate and safer comparisons 



Pop Quiz

Which operator is used to find the remainder in JavaScript?

A

/

B

%

Pop Quiz

Which operator is used to find the remainder in JavaScript?

A

/

B

%

The top corners of the slide feature decorative white circuit-like lines on a dark blue background. These lines start from the left and right edges, curve upwards and inwards, and then branch out horizontally towards the center, ending in small circles. The lines are layered, creating a sense of depth and connectivity.

Hands-On | declare variables and print values, perform arithmetic operations and compare values and print result

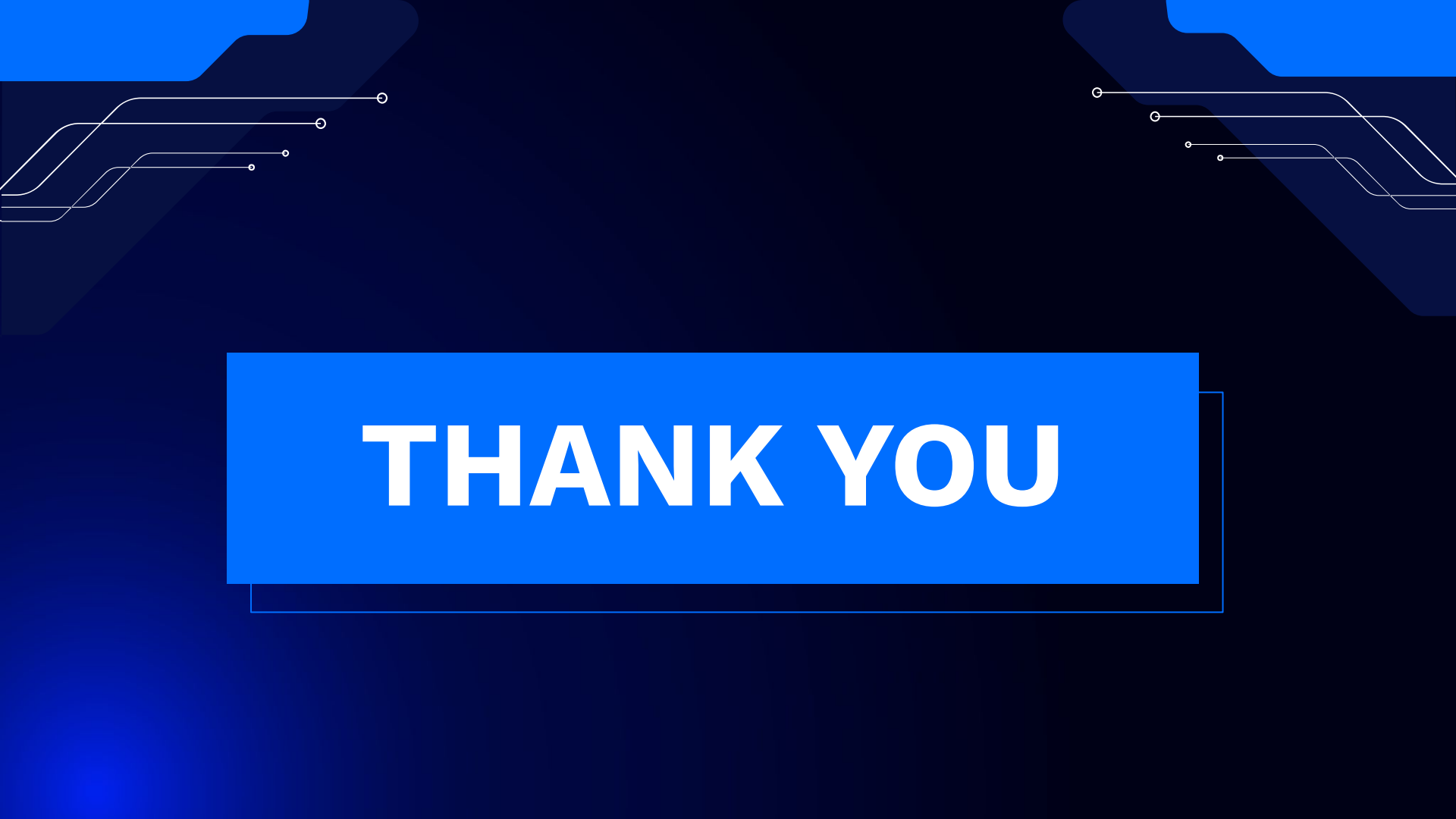
Summary

- JavaScript is used to add logic and interactivity to web pages
- It runs in the browser and can also run on the server (Node.js)
- Programs use values and data types like number, string, and boolean
- Variables store data and are declared using let, const, or var
- Code follows steps like declare → assign → print
- Identifiers are variable names and must follow specific rules
- Comments help explain code and improve readability
- Operators (arithmetic & comparison) are used for calculations and decisions
- Understanding == vs === ensures correct comparisons





DOUBT RESOLUTION



THANK YOU